

Here is the fully updated end to end project doc. It supersedes the older draft and reflects the newer direction: **preservation first, narrow gap filling second, polish third**, with Gemini used only where it is genuinely additive instead of forced.

Elevator pitch, judge version

Imagine every iconic sports moment becoming a place you can step inside. Instead of watching one flat clip of a buzzer beater, game winner, or poster dunk, you upload every angle people captured and instantly get the moment back as a living 3D scene. You can pause at the exact release, orbit around the athlete, scrub backward and forward in time, drop to court level, and then switch into a director mode that automatically turns that same reconstructed moment into a cinematic replay. This is not an AI highlight editor. It is a sport agnostic replay engine that reconstructs the play itself, intelligently patches missing viewpoints, and lets you experience a moment from angles that were never cleanly captured in the first place. ([Human Sensing Lab](#))

One sentence version

Upload multiple videos of the same sports moment, get back a fully interactive 3D replay plus a cinematic director cut generated from the same patched scene. ([Human Sensing Lab](#))

What we are actually building

This is **one reconstruction engine with two output modes**, not two unrelated products.

1. Interactive Replay

This is the main product and the thing the judges should actually touch. It is a refined, patched, navigable 4D replay of the moment itself. The judge can orbit the camera, zoom, scrub through time, slow down, speed up, and inspect the play from different viewpoints. Crucially, this is **not** the raw first-pass reconstruction. By the time a judge sees it, it has already gone through gap detection, targeted densification, and cleanup. ([GitHub](#))

2. Director Mode

This is not a separate AI hallucination. It is an autonomous cinematic path **over the same patched scene**. Instead of the judge manually controlling the camera, the system choreographs the replay like a high-end broadcast package, orbiting, zooming, pausing at the key frame, slowing down the apex, then re-accelerating out of the moment. Director Mode exists to make the result feel premium and unforgettable, but it still rides on top of the same reconstructed scene rather than replacing it. ([Human Sensing Lab](#))

The core technical thesis

The project only works if we respect one hard rule:

Do not hand the whole problem to a generative model.

The right way to do this is:

1. reconstruct as much real geometry and motion as possible from the real videos
2. identify the weak angular sectors and weak timestamps
3. synthesize only a few missing intermediate views in those weak sectors
4. feed those repaired views back into the scene refinement loop
5. polish the final replay after the geometry is already mostly solid

That logic is directly aligned with recent work like BulletGen, which uses generative models to correct errors and complete missing information in a Gaussian-based dynamic scene at a frozen bullet-time step, and GS-GS, which improves sparse-view Gaussian Splatting by generating unseen views and iteratively refining consistency rather than replacing the scene outright. ([arXiv](#))

So the winning framing is:

This is a measured reconstruction system with targeted AI densification, not a video generator pretending to be reconstruction. ([arXiv](#))

Exactly which Gemini models to use

Use **two** Gemini models in the judged core flow:

A. Gemini Live API

This should be the **Replay Director**, meaning the intelligence and control layer that sits on top of the whole system. The Live API is built for low-latency, real-time voice and vision interaction over continuous streams of audio, images, video, and text, and Gemini function calling lets it trigger backend tools instead of just chatting. That makes it perfect for a judge-facing assistant that understands the clip set, explains what the system is doing, and controls the replay in real time. ([Google AI for Developers](#))

What Live should do:

- accept preview clips or thumbnails from the upload set
- help identify the key sports moment semantically
- explain which angles are strongest

- call backend functions like `analyze_clips`, `start_sync`, `run_reconstruction`, `jump_to_keyframe`, `toggle_director_mode`, `orbit_camera`, `show_gap_confidence`
- narrate the replay naturally while the judge interacts with it ([Google AI for Developers](#))

B. Gemini 2.5 Flash Image, also known as Nano Banana

This should be used for **surgical visual repair**, not broad generation. Google describes it as a fast image generation and editing model that supports conversational image editing, low-latency workflows, targeted transformations, and multi-image fusion. That makes it much better suited than Veo for your actual need, which is to patch a small number of carefully chosen intermediate views using adjacent anchor views and masks. ([Google Cloud Documentation](#))

What Nano Banana should do:

- take the left anchor view and right anchor view around a weak angular gap
- optionally take the ugly center render from the measured replay
- take a mask that says which regions are broken
- generate a repaired intermediate still view that preserves as much of the measured scene as possible
- produce only a handful of these key repaired views, which are then fed back into refinement rather than shown raw as the final answer ([Google Cloud Documentation](#))

Why Veo is not in the critical path

Veo is excellent at high-fidelity video generation and supports first/last frame conditioning and reference images, but it is still a video generation model, not a geometry-preserving reconstruction model. Because your core requirement is to preserve the carefully reconstructed 4D scene and only fill very narrow missing gaps, Veo is too unconstrained for the judged core experience. I would keep it out of the main flow. At most, it becomes an optional marketing trailer or post-demo extra, not the heart of the product. ([Google AI for Developers](#))

Full end to end system architecture

Stage 1. Input ingestion and preprocessing

Do **not** force Gemini here if deterministic tooling works better.

Use a custom ingestion pipeline first:

- `ffmpeg` for trimming, frame extraction, fps normalization, and resolution normalization
- audio waveform correlation for coarse sync

- optional visible sync cues when you control the capture
- OpenCV-based quality checks for blur, shakiness, and usable frame density

The goal of this stage is not intelligence. It is to turn a messy clip set into a clean, aligned dataset that the geometry stack can actually use. The semantic layer comes after this, not before it. This is the right place to be boring and reliable. The need for accurate multiview alignment before dense reconstruction is exactly why SfM and MVS pipelines like COLMAP exist. ([GitHub](#))

Stage 2. Semantic event understanding

Once the inputs are normalized, use **Gemini Live** to help determine the moment that matters and make the system feel smart.

Live can inspect preview clips and help answer questions like:

- which clips actually contain the play
- what the key event is
- which camera angles are likely most useful
- which moment should be the hero frame

This keeps Gemini in a role where it is strongest, namely understanding and interacting over multimodal streams, instead of doing geometry or brute-force preprocessing. ([Google AI for Developers](#))

Stage 3. Camera pose recovery

Use **hloc + COLMAP** as the geometry front end.

hloc is a modular toolbox for state-of-the-art visual localization that combines image retrieval, feature matching, and Structure-from-Motion workflows, and COLMAP is a general-purpose SfM and MVS pipeline that works on ordered or unordered image sets. This pair is still the most grounded way to estimate camera poses, sparse structure, and scene consistency from a multiview clip set. ([GitHub](#))

Outputs of this stage:

- camera intrinsics when recoverable
- camera extrinsics
- sparse scene structure
- view overlap confidence
- a rough sense of which angular sectors are weak ([GitHub](#))

Stage 4. Measured scene reconstruction

Use **4DGaussians** as the main reconstruction backbone.

4DGaussians is still the most practical public starting point for dynamic Gaussian Splatting in this context because the public repo is real, it documents training, and it supports custom multiview data pipelines. This is your main scene representation. It gives you the base 4D replay that will later be patched and refined. ([GitHub](#))

At this point, the output is:

- a first-pass 4D scene
- renderable novel views
- enough structure to see where the replay is solid and where it breaks ([GitHub](#))

Stage 5. Sports-specific compute strategy

Borrow the **LiveSplats** idea, not necessarily the entire codebase.

LiveSplats is explicitly about 3D live streaming of sports events from multiview video, and its key system insight is extremely relevant here: treat the moving subjects and the environment differently, and parallelize the reconstruction workload across workers or GPUs where possible. ([Human Sensing Lab](#))

In your build, that means:

- player and ball are high priority
- background court and crowd get cheaper treatment
- refinement jobs can run separately from the viewer
- the UI should never block on a heavy optimization job ([Human Sensing Lab](#))

Stage 6. Gap detection

Now that you have a measured 4D scene, compute a **gap map**.

The system should determine:

- which timestamps render well
- which timestamps are unstable
- which angular sectors are under-observed
- which specific regions of those views break first

This is where your product gets much smarter. Instead of saying “the replay looks bad somewhere,” you now know exactly **where** the replay is weak and **when** it is weak. That lets

you patch with precision instead of brute force. This logic is consistent with BulletGen and GS-GS, both of which use generated content to reinforce under-observed parts of a Gaussian scene rather than regenerating everything. ([arXiv](#))

Stage 7. BulletGen-style densification, this is core

This is where the project becomes special.

Use **BulletGen as the conceptual core** for the enhancement loop. BulletGen's main contribution is exactly what you need: use generative output at a frozen bullet-time step to correct missing information and supervise the optimization of the Gaussian scene. In other words, the generated views are not the product. They are **supporting evidence** that improves the scene itself. ([arXiv](#))

Your implementation should follow that philosophy:

1. choose a key timestamp, usually the release point, apex, impact frame, or trick peak
2. choose a narrow weak angular sector, like a missing 10 to 15 degree band
3. render the nearest reliable left and right anchor views from the measured replay at that exact timestamp
4. optionally include the ugly center render if it exists
5. use **Nano Banana** to repair only the broken regions of 1 or 2 intermediate views between those anchors
6. feed those repaired intermediate views back into scene refinement so the **interactive replay itself** improves

This is the single most important update to the old plan. The repaired views are not just for a cinematic mode. They are used to make the **actual interactive replay** better. ([Google Cloud Documentation](#))

Stage 8. Temporal cleanup

Once the scene has been spatially improved, use dedicated video cleanup tools for temporal quality.

Use **RGVI** for localized video hole repair and reference-guided inpainting. The official repo presents it as an AAAI 2025 video inpainting method focused on elevating flow-guided inpainting with reference generation, which is exactly the kind of targeted temporal cleanup you want after spatial repair. ([GitHub](#))

Use **RIFE** after that for frame interpolation and smoother motion. The public repo explicitly reports 30+ FPS for 2X 720p interpolation on a 2080 Ti and supports arbitrary-timestep interpolation, making it a practical final polish step. ([GitHub](#))

So the cleanup logic is:

- BulletGen-style densification for structural gaps
- Nano Banana for narrow still-view repairs
- RGVF for local temporal consistency
- RIFE for final smoothness and premium feel ([arXiv](#))

Stage 9. Product surfaces

Interactive Replay, the main judged surface

This is what the judge interacts with directly:

- scrub forward and backward in time
- orbit camera freely within safe bounds
- zoom in and out
- jump to the hero frame
- switch between baseline and refined view if you want to show the improvement

This replay should already be the **patched and refined** version of the scene, not the raw output. That is a major product distinction. The judge should feel like they are touching a finished interactive object, not a research artifact. ([arXiv](#))

Director Mode, the cinematic judged surface

This mode runs on the same refined scene and automatically:

- chooses a strong opening angle
- eases into the hero frame
- slows down at the peak moment
- orbits through the newly repaired sector
- zooms for dramatic emphasis
- exits cleanly

Director Mode is there to make the result emotionally memorable. It is not there to hide the reconstruction. It should feel like a premium layer on top of a system that already works. ([Human Sensing Lab](#))

Where Gemini Live appears in the final demo

Live should be visible and central, but not overloaded.

The judge experience should look like this:

- upload or select the clip set
- Live says what moment it identified and what the system is doing
- Live can answer “show me the release point,” “orbit from the weak side,” “play director mode,” “why is this angle stronger,” and similar commands
- Live calls backend functions to make those things happen

This makes Live genuinely core to the product rather than a bolt-on chatbot. Because the Live API is designed for low-latency voice and vision interaction and Gemini function calling is meant to bridge the model to external tools, this is exactly the kind of integration that makes sense for your track strategy. ([Google AI for Developers](#))

Why this is the best prize strategy

Given the judging style you described, the right optimization is:

- build a genuinely great overall product first
- use Google models in places where the judges can clearly see they matter
- avoid superficial usage that only exists to check a box

That is why the best two Gemini models are:

- **Live**, because it becomes the real-time product intelligence and control layer
- **Nano Banana**, because it becomes the narrow, high-value repair tool that helps improve actual replay quality

This combination gives you a credible shot at **Best Overall** while also making your Google model usage feel substantial enough to matter for the model-specific tracks. Veo and Lyria are easier to notice, but much easier to waste in this particular product. ([Google AI for Developers](#))

Optional branch, only after the core loop works

If the core pipeline works early, test **4D-Rotor Gaussian Splatting** as a motion-quality or performance branch. NVIDIA’s project describes it as a dynamic-scene method based on anisotropic 4D XYZT Gaussians, and the paper reports very high inference FPS. However, the GitHub repo also states that the optimized CUDA framework behind the paper’s headline FPS numbers is not being released, and that the public code is a PyTorch research implementation. That makes it a high-upside experiment, not the thing to bet the whole hackathon on. ([NVIDIA](#))

The actual build order

Phase 1. Prove the measured loop once

- controlled capture

- preprocessing
- sync
- hloc + COLMAP
- 4DGaussians
- basic novel view replay

Success criterion: a crude but real interactive replay exists. ([GitHub](#))

Phase 2. Add gap awareness

- compute weak sectors
- identify the hero timestamp
- build the gap/confidence map

Success criterion: you know exactly where and when the replay is bad. ([arXiv](#))

Phase 3. Add BulletGen-style repair

- generate 1 or 2 repaired intermediate views at the hero timestamp
- feed those back into scene refinement
- improve the actual interactive replay

Success criterion: the interactive replay itself becomes visibly stronger around the weak sector. ([arXiv](#))

Phase 4. Add temporal polish

- RGVI
- RIFE
- final smoothing

Success criterion: the replay feels much cleaner and less jittery. ([GitHub](#))

Phase 5. Add product surfaces

- refined interactive replay
- director mode
- Live-powered Replay Director

Success criterion: the demo now feels like a product, not a pipeline. ([Google AI for Developers](#))

What to cut if time gets tight

Cut in this order:

1. optional 4D-Rotor branch
2. extra sports examples
3. fancy UI styling
4. any Veo trailer idea
5. broad clip-ingestion intelligence beyond the one demo set

Do **not** cut:

1. measured replay loop
2. gap detection
3. BulletGen-style refinement logic
4. interactive replay controls
5. Live integration for the judge-facing experience ([arXiv](#))

Final judge-facing story

“We take multiple videos of the same sports moment and reconstruct the play itself as a navigable 3D scene. Then we find the exact viewpoints the cameras missed, intelligently densify only those gaps, feed that information back into the reconstruction, and let you explore the final replay in real time. On top of that, our Replay Director turns the same refined scene into a cinematic broadcast-style cut.”

That is the strongest version of your idea because it is:

- easy to understand
- visually insane
- technically deep
- honest about what is reconstructed and what is repaired
- perfectly aligned with the most innovative parts of the stack you researched ([Human Sensing Lab](#))